



Institut Supérieur d'informatique et
de Mathématiques de Monastir



Université de Monastir

Système d'exploitation 2

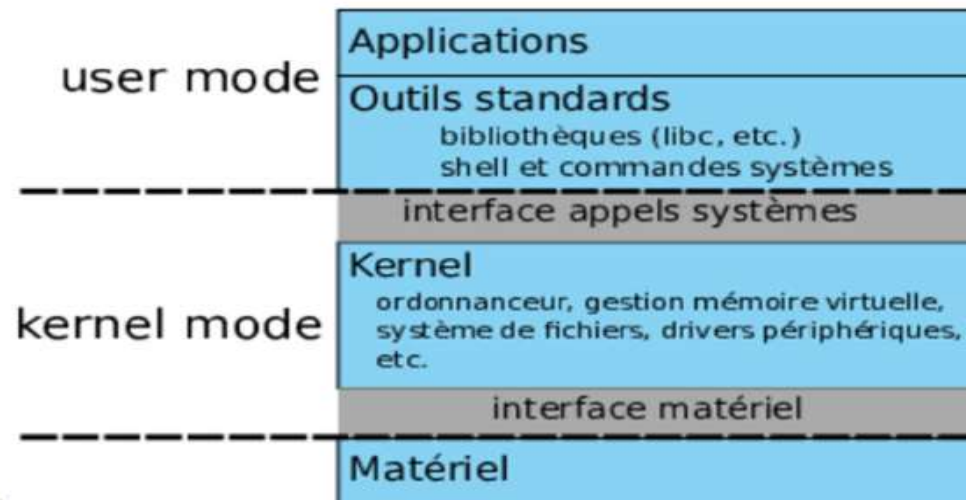
RIM MEGDICHE
rim.magdich@gmail.com

Chapitre 1

Les appels systèmes

Modes de fonctionnement: Noyau Vs utilisateur

- Le mode noyau et le mode utilisateur sont deux états de fonctionnement qui définissent le niveau d'accès et de contrôle qu'un programme sur le matériel et les ressources.
- Ces modes de fonctionnement visent à assurer la protection du système d'exploitation contre les intrusions et les erreurs,
- **le mode noyau** pour le système d'exploitation, où toutes les instructions sont autorisées.
- **le mode utilisateur** pour les programmes des utilisateurs et les utilitaires, où certaines instructions ne sont pas permises.



Les appels système

Comment passer en mode noyau ?

2 méthodes:

- **Interruption**
 - interruption logicielle
 - Générée par le processeur en exécutant une instruction
 - division par zéro, accès mémoire illicite
 - interruption générée par le matériel (IRQ)
 - “Je viens de recevoir un message.” – la carte réseau
 - “J’ai fini de copier les données sur le disque dur.” – le moteur DMA
- **Appel système**
 - l'utilisateur demande au SE un service

Les appels système

Que ce qu'un appel système ?

- Un appel système (en anglais, **system call**, abrégé en **syscall**) est une fonction fournie par le noyau et utilisée par les programmes s'exécutant dans l'espace utilisateur.(tous les programmes distincts du noyau).
- Ce système permet de contrôler de façon sécurisée les applications dans l'espace utilisateur.
- Quelques appels systèmes classiques :
 - **open, read, write et close** qui permettent les manipulations sur les systèmes de fichiers,
 - **alloc, free** pour allouer et désallouer de la mémoire.

Les appels système

Que ce qu'un appel système ?

- Les appels système peuvent être invoqués via un interpréteur de commandes (ex shells), une interface graphique, des utilitaires ou encore des programmes d'application.

Exemple: Le programme C suivant utilise les appels système open, write et read. Il crée un fichier dans lequel il copie les données lues à partir du clavier.

```
#include <unistd.h> // pour open, write, read
#include <fcntl.h> // O_CREAT, O_WRONLY, O_TRUNC
#define taille 80
int main ( )
{ int fd , nbcar;
  char buf[taille] ;
  //créer/ouvrir un fichier
  fd = open("fich", O_CREAT| O_WRONLY | O_TRUNC);
  if(fd==-1)
  { write(2, "Erreur d'ouverture \n", 20);

    return -1 ;
  }
  write(1, "Ouverture avec succès \n" , 23);

  // écrire dans fich les données lues à partir du clavier
  while ((nbcar = read(0, buf, taille)) > 0)
    if( write(fd, buf, nbcar) == -1) return -1 ;
  return 0 ;
}
```

Les appels système

- Un appel système consiste en une interruption logicielle qui a pour rôle d'accéder au noyau du système d'exploitation :
 1. changer le mode d'exécution pour passer du mode utilisateur au mode noyau (exemple, sous Linux: `int 0x80` ou `syscall`);
 2. récupérer les paramètres et vérifier la validité de l'appel;
 3. exécuter la fonction demandée;
 4. récupérer la (les) valeur(s) de retour;
 5. retourner au programme appelant avec retour au mode utilisateur.

Les appels système

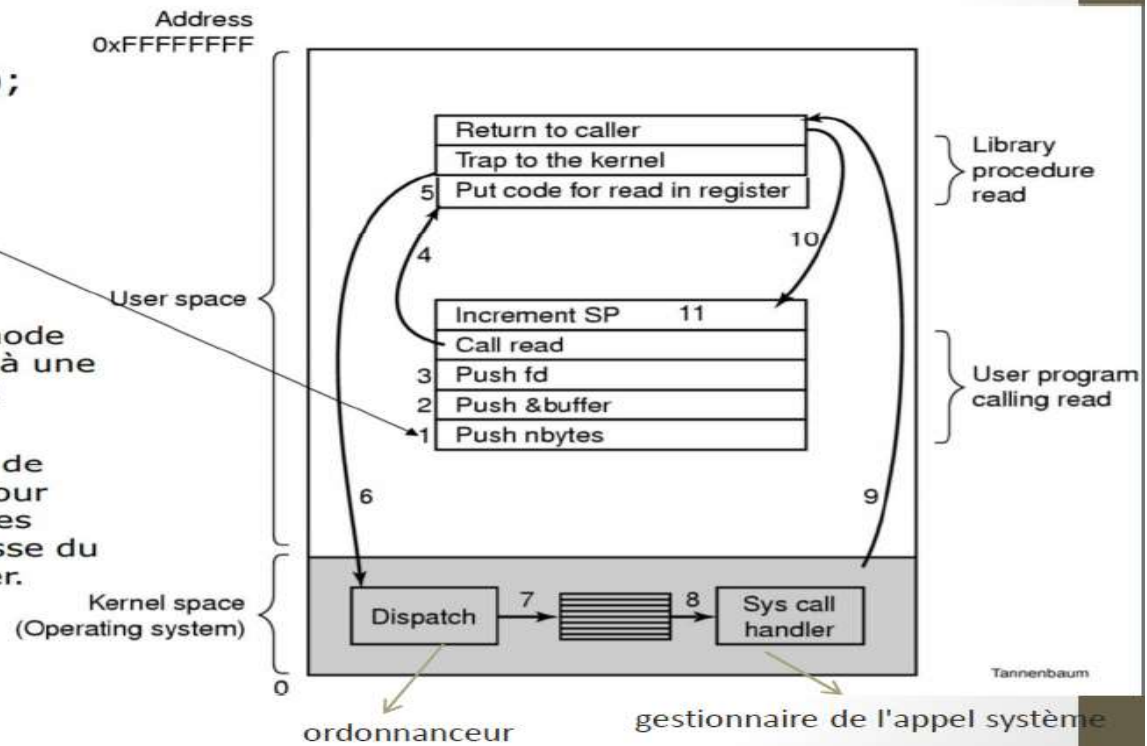
Que se passe-t-il lors d'un appel système ?

Exemple:

`read(fd,buffer,nbytes);`

Étape 6 : bascule vers le mode noyau puis se branche à une adresse fixée du noyau

Étape 7 : utilise le numéro de l'appel système `read` pour récupérer de la table des appels système, l'adresse du code de `read` à exécuter.



SP: Stack Pointer (pointeur de pile)

Les appels système

POSIX et Win32 (Windows)

POSIX	Win32	Description
fork	CreateProcess	Créer un nouveau processus / fork + execve
waitpid	WaitForSingleObject	Attendre la fin d'un processus
execve		Remplacer l'exécutable du processus
exit	ExitProcess	Terminer le processus
open	CreateFile	Créer ou ouvrir un fichier
close	CloseHandle	Fermer un fichier
read	ReadFile	Lire d'un fichier
write	WriteFile	Ecrire dans un fichier
lseek	SetFilePointer	Changer la position courante dans le fichier
stat	GetFileAttributesEx	Récupérer les attributs d'un fichier
mkdir	CreateDirectory	Créer un répertoire
rmdir	RemoveDirectory	Supprimer un répertoire vide
link		Ajouter un lien physique vers un fichier
unlink	DeleteFile	Supprimer un lien physique vers un fichier
mount		Ajouter un système de fichiers à l'arborescence de fichiers
umount		Retirer un système de fichiers
chdir	SetCurrentDirectory	Changer de répertoire de travail (courant)
chmod		Changer les permissions d'accès d'un fichier
kill		Envoyer un signal à un processus
time	GetLocalTime	Obtenir le temps courant

man syscalls

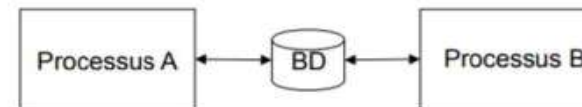
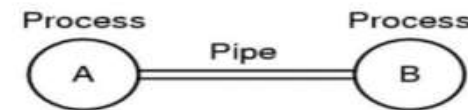
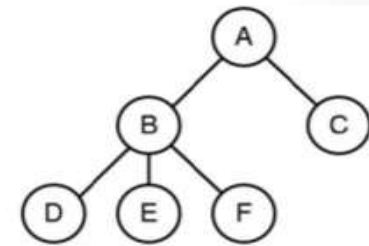


Les appels système

Appels système : gestion de processus

- Les appels système permettent, notamment, :

- la création, la terminaison de processus,
- la communication interprocessus (segments de données partagés, fichiers, tubes de communication (pipes), etc.),
- la synchronisation (éviter les accès simultanés lecture/ écriture ou écriture/écriture à une même donnée)
- Et autre...



Les appels système

Les Processus Linux : PID, Groupes et Hiérarchie

PID (Process ID) : Identifiant unique de chaque processus.

PPID (Parent PID) : Identifiant du processus qui a créé ce processus (son "père").

GID (Group ID) : Identifiant du groupe de processus auquel appartient le processus.

Règle clé : Deux processus ayant le même PPID appartiennent généralement au même groupe (même GID).

La création et la terminaison d'un processus (Linux)

Les deux actions les plus importantes pour un processus sont sa création et sa terminaison :

1. **Création** : le processus, interpréteur de commande de linux, crée un nouveau processus à chaque fois que l'utilisateur saisit une nouvelle commande.
 - le processus en cours, appelé processus père, utilise l'appel système **fork()** pour créer un clone de lui-même, appelé processus fils.
 - Le code du processus fils est changé ; on utilise pour cela l'une des variantes de l'appel système **exec()**.
2. **Terminaison**: Un processus se termine automatiquement lorsqu'il finit l'exécution de sa fonction **main()**.

Création d'un processus

Le mécanisme de duplication

Un nouveau processus est créé via l'appel système **fork()**.
fork() crée une copie quasi conforme du processus appelant.

Résultat après **fork()** :

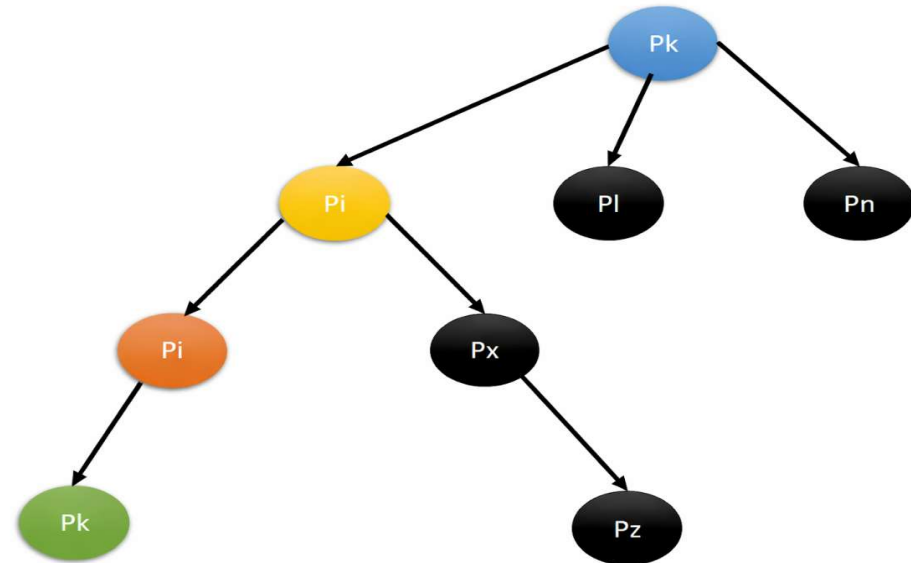
- **Processus Père (Parent)** : Reprend son exécution.
- **Processus Fils (Child)** : Nouveau processus, commence son exécution.

Père et fils s'exécutent en parallèle.

Création d'un processus

De fork() à l'Arbre des Processus

- Un père peut créer plusieurs fils (appels fork() successifs).
- Un fils peut lui-même créer ses propres processus (en appelant fork()).
- Cela génère rapidement une hiérarchie en arbre de profondeur variable.
- Commande pour visualiser : **ps tree**



Création d'un processus

L'Appel Système fork()

Crée un clone du processus appelant



```
#include <unistd.h>
#include <sys/types.h>

pid_t fork(void);
```

- **Type de retour** : pid_t (type entier pour les identifiants de processus)
- **Paramètres** : Aucun (void)
- **Fonction** : Crée un nouveau processus en dupliquant le processus appelant

Création d'un processus

L'Appel Système fork()

Valeurs de retour :

0	Dans le processus FILS
PID > 0	Dans le processus PÈRE (PID du fils)
-1	En cas d'ERREUR (errno positionné)



Ceci permet de différencier, à l'exécution, le contexte du père de celui du fils.

Les causes de l'échec sont les suivantes :

- Soit que le nombre maximal de processus pour l'utilisateur en cours, ou pour le système a été atteint ;
- Soit encore que le noyau n' a pas pu allouer suffisamment de mémoire pour créer le nouveau processus.

Exemple Complet avec fork()

```
pid_t nouv_pid;  
nouv_pid = fork();  
  
if (nouv_pid == -1) {  
    printf("Erreur de création\\n");  
}  
else if (nouv_pid == 0) {  
    // FILS  
    printf("Je suis le fils: pid=%d\\n", getpid());  
    printf("Mon père: ppid=%d\\n", getppid());  
}  
else {  
    // PÈRE  
    printf("Je suis le père: pid=%d\\n", getpid());  
    printf("Mon fils: %d\\n", nouv_pid);  
}
```

Exemple Complet avec fork()

Le programme suivant :

```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>

int main()
{
    pid_t nouv_pid;

    nouv_pid=fork();
    if(nouv_pid== -1)
        printf("erreur de creation \n");
    else if(nouv_pid==0)
        printf(" je suis le fils : pid=%d \n ", pid) ;
    else
        printf(" je suis le père : pid=%d \n ", pid) ;
    return 0 ;
}
```

Permet de créer un processus fils. Son exécution donne, par exemple :

```
je suis le père : pid=856
je suis le fils : pid=0
```

Ce qui montre que les deux processus, père et fils, sont bien en train d'être exécuté. L'ordre de l'affichage dépend de l'ordonnaceur.

Copy-on-Write (COW)



Optimisation majeure pour éviter la recopie inutile

- 1 Après `fork()` : père et fils partagent la même mémoire physique
- 2 Accès en lecture seule : aucune duplication nécessaire
- 3 À la première écriture : duplication de la page mémoire modifiée
- 4 Résultat : gain considérable en temps et en mémoire

Seule la pile est dupliquée lors du `fork()`

Les appels système

Exercice

Exercice : Création d'un processus fils qui fait un calcul

But :

- Le père demande un nombre à l'utilisateur.
- Le fils calcule le **carré** de ce nombre.
- Le père continue et affiche juste son PID.

Les appels système

Correction

```
#include <stdio.h>      // Entrées/sorties standard
#include <unistd.h>      // fork(), getpid(), getppid()
#include <sys/types.h>  // Types pour les processus

int main() {
    int nombre;
    printf("Entrez un nombre : ");
    scanf("%d", &nombre);

    int x = fork();  // * POINT CLÉ : Création du processus fils

    if (x == 0) {
        // Code du fils
        printf("Fils : le carré de %d est %d\n",
            nombre, nombre * nombre);
        printf("Fils PID = %d, Père PID = %d\n",
            getpid(), getppid());
    }
    else if (x > 0) {
        // Code du père
        printf("Père : je continue mon programme. Mon PID = %d\n",
            getpid());
    }
    else {
        printf("Erreur ! Impossible de créer un fils.\n");
    }

    return 0;
}
```

Création d'un processus

L'Appel Système `execve()`

C'est quoi `execve()` ?

- `execve()` est un appel système qui sert à remplacer le programme en cours par un autre programme.

 Point clé :



`execve()` ne crée pas de nouveau processus.



Il change le code du processus courant.

Création d'un processus

L'Appel Système `execve()`

Pourquoi `execve ()` avec `fork ()`?

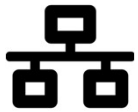
En Linux, on crée un nouveau processus en **deux étapes** :

1. `fork()` → crée un **processus fils**
2. `execve()` → le **fils charge un nouveau programme**

C'est exactement ce que fait le **shell**.

Création d'un processus

L'Appel Système execve()



Remplace l'image du processus par un nouveau programme

```
#include <unistd.h>

int execve(const char *filename, char **argv, char **envp);
```

Paramètres :

- filename : chemin du fichier exécutable
- argv : tableau d'arguments (se termine par NULL)
- envp : variables d'environnement (se termine par NULL)



Important : Si execve() réussit, il n'y a JAMAIS de retour !

Création d'un processus

L'Appel Système `execve()`

Les Variantes de `exec()`

`execl()`

Liste d'arguments

`execle()`

Liste + environnement

`execvp()`

Tableau + recherche PATH

`execlp()`

Liste + recherche PATH

`execv()`

Tableau d'arguments

`execve()`

Véritable appel système (base)

Pattern classique : `fork()` + `exec()`

```
if (fork() == 0) {  
    execl("/bin/ls", "ls", "-l", NULL);  
    // Cette ligne n'est JAMAIS exécutée si exec réussit  
}
```

Les appels système

Chargement/changement d'un exécutable avec exec()

- La primitive `exec()` servira à un processus clone d'exécuter le code d'un autre programme à sa place,
- un processus peut donc changer de code par un appel à `exec()` sans changer le numéro de PID.
- Il y a plusieurs fonctions de la famille `exec()` qui sont légèrement différentes,

Exemple : fichier `test.c` compilé avec GCC

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>

int main()
{ execl("/bin/ls", "ls", "-l", NULL);
  return 0;
}
```

Exemple d'exécution :

```
total 20
-rwxrwxr-x 1 Etudiant LFSI 8551 Oct 8 12:35 test
-rw-rw-r-- 1 Etudiant LFSI 139 Oct 8 12:35 test.c
-rw-rw-r-- 1 Etudiant LFSI 139 Oct 8 12:35 test.c~
```

Les appels système

Écrire un programme C qui :

1. Crée un processus enfant avec `fork()` .
2. Si le processus est l'enfant :
 - Affiche "Je suis le fils, je vais exécuter /bin/ls".
 - Utilise `execve()` pour exécuter la commande `ls -l /` .
3. Si le processus est le parent :
 - Affiche "Je suis le père, j'attends que le fils termine...".

Indice pour `execve()` :

```
char *argv[] = { "ls", "-l", "/", NULL };  
char *envp[] = { NULL }; // variables d'environnement  
execve("/bin/ls", argv, envp);
```

Les appels système

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        // Erreur
        perror("fork");
        return 1;
    }
    else if (pid == 0) {
        // Processus enfant
        printf("Je suis le fils (pid=%d), je vais exécuter /bin/ls\n", getpid());
        char *argv[] = { "ls", "-l", "/", NULL };
        char *envp[] = { NULL };
        execve("/bin/ls", argv, envp);
    }
}
```

```
// Si execve échoue
    perror("execve");
    return 1;
}
else {
    // Processus parent
    printf("Je suis le père (pid=%d), j'attends que le fils
termine...\n", getpid());
    wait(NULL);
    printf("Le fils a terminé\n");
}

return 0;
}
```

Les appels système

L'Appel Système wait()

Synchronisation des processus entre père et fils

Pourquoi synchroniser les processus ?

- Le père et le fils s'exécutent **en parallèle** après `fork()`
- **Problème** : Le père peut terminer avant ou après son fils
- **Conséquence** : Risque de créer des processus **zombies** ou **orphelins**
- **Solution** : L'appel système `wait()` pour synchroniser la terminaison

Les appels système

L'Appel Système wait()

- L'appel système **wait ()** et **exit()** permettent de synchroniser les processus père et fils.
- **exit(entier)** : la fin d'un processus fils après un exit, la valeur de l'entier est transmis au code de retour du processus père.
 - Une bonne fin d'un code, donne un code de retour **NULL** par défaut.
- **wait (*entier)**: *entier : est l'adresse ou l'on peut récupérer la valeur de terminaison du processus fils.
 - Remplacer *entier par **NULL** si l'on ne veut pas récupérer la valeur de terminaison.

Les appels système

L'Appel Système wait()

Terminaison d'un fils

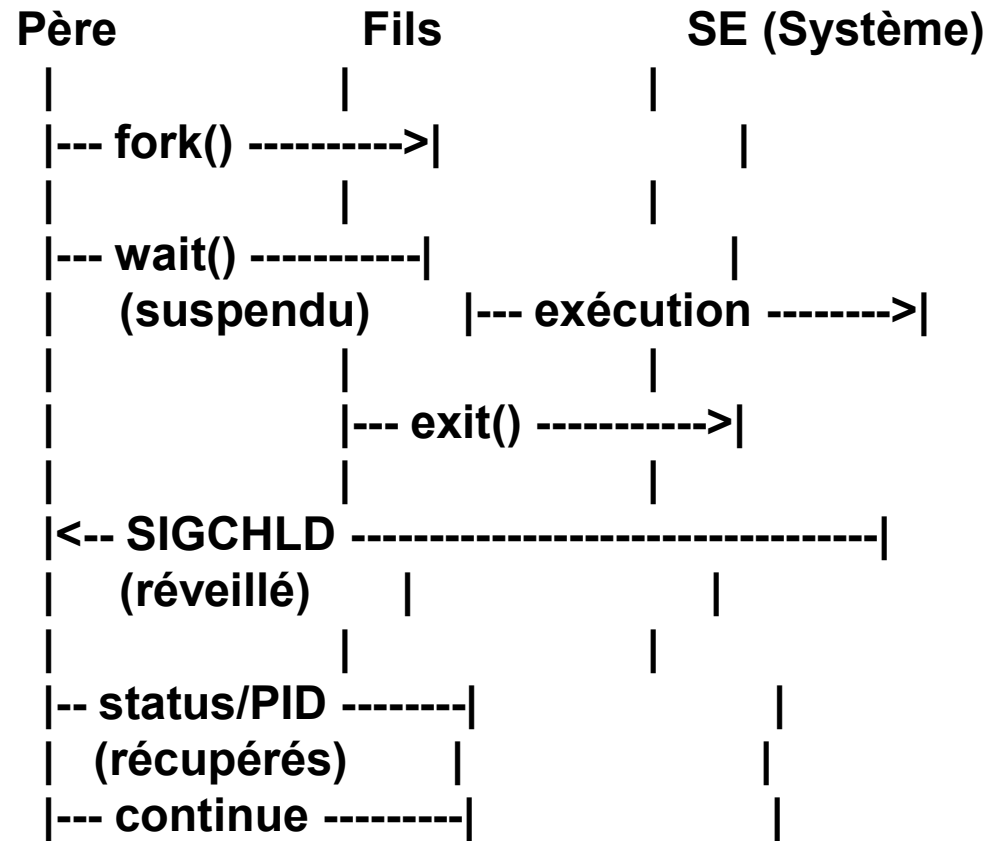
Comment le père est notifié ?

1. **Terminaison du fils** : Le processus fils se termine (normalement ou anormalement)
2. **Signal du SE** : Le Système d'Exploitation envoie un signal (**SIGCHLD**) au père
3. **Réveil du père** : Le signal "réveille" le processus père en attente
4. **Récupération d'information** : Le père reçoit via `wait()` :
 - **PID** du fils terminé
 - **Statut** de terminaison (code retour ou signal)

Les appels système

L'Appel Système wait()

Terminaison d'un fils



Les appels système

L'Appel Système wait()

Exemple d'exécution :

```
je suis le fils de pid:30
je suis le père de pid: 29 pid de fils 30 d adresse:
212
```

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
int main()
{ int x,y,z;
  if ((x=fork())==0)
  { printf("je suis le fils de pid:
%d\n",getpid());
    exit(3); }
  else if (x==-1){
    printf("Erreur lors de création de fils! ");}
  else { /*le père doit attendre la fin de son
fils*/
    y=wait(&z);
    printf("je suis le père de pid: %d\n pid du
fils: %d\n
d adresse: %d\n", getpid(), y, z); }
  return 0; }
```

Les appels système

L'Appel Système wait()

Les fonctions **WEXITSTATUS** et **WIFEXITED**

Quand un processus enfant se termine, il transmet un statut de terminaison à son parent via **wait()**. Ce statut est un nombre encodé qui contient plusieurs informations. Pour le décoder proprement, on utilise deux macros :

1. **WIFEXITED(wstatus)**

Fonction : Vérifie si le processus s'est terminé **normalement**.

Comment ? :

Retourne **vrai (non nul)** si l'enfant a utilisé `exit()` ou a terminé son `main()` avec un `return`.

Retourne **faux (0)** si l'enfant a été tué par un signal (ex: `SIGKILL`, `SIGSEGV`).

Utilisation : Toujours appeler d'abord cette fonction pour savoir si le processus a eu une fin normale.

Les appels système

L'Appel Système wait()

Les fonctions **WEXITSTATUS** et **WIFEXITED**

Quand un processus enfant se termine, il transmet un statut de terminaison à son parent via **wait()**. Ce statut est un nombre encodé qui contient plusieurs informations. Pour le décoder proprement, on utilise deux macros :

2. **WEXITSTATUS(wstatus)**

Fonction : Récupère le **code de retour réel** du processus.

Condition : Ne doit être utilisé **que si** **WIFEXITED(wstatus)** est vrai.

Valeur retournée : Un entier entre 0 et 255.

Les appels système

L'Appel Système wait()

Les fonctions **WEXITSTATUS** et **WIFEXITED**

Pourquoi cette complexité ?

Parce que dans wstatus (ex: 212), il y a PLUSIEURS informations cachées :

Bit 1 : "Normal ou pas ?" → lu par **WIFEXITED()**

Bits 2-9 : "Le vrai code" → lu par **WEXITSTATUS()**

Autres bits : D'autres infos techniques

Les appels système

L'Appel Système wait()

Les fonctions **WEXITSTATUS** et **WIFEXITED**

Pourquoi cette complexité ?

Parce que dans wstatus (ex: 212), il y a PLUSIEURS informations cachées :

Bit 1 : "Normal ou pas ?" → lu par **WIFEXITED()**

Bits 2-9 : "Le vrai code" → lu par **WEXITSTATUS()**

Autres bits : D'autres infos techniques

Les appels système

L'Appel Système wait()

Les fonctions **WEXITSTATUS** et **WIFEXITED**

Démarche

Étape 1 : Le fils termine son travail `exit(42);` // Et dit « je termine avec le code 42 »

Étape 2 : Le père attend et reçoit un code `int wstatus;`
`wait(&wstatus);` // Le père attend et reçoit un reçu "wstatus"

Maintenant, **wstatus** ressemble à un **papier codé**. Exemple : 212 dans ce cas.

Étape 3 : Comment lire ce papier codé ?

I. "Est-ce qu'il a terminé normalement ?"

```
if (WIFEXITED(wstatus)) {  
    // OUI, il a fini proprement (avec exit() ou return)  
    // On peut passer à la question 2  
} else {  
    // NON, il s'est fait interrompre (par un signal, un crash)  
    printf("Il s'est fait arrêter en chemin !\n");  
}
```

II. "Quel était son score final ?"

```
int score = WEXITSTATUS(wstatus);  
// score = 42 (le vrai code, pas 212)  
printf("Il a terminé avec le score : %d\n", score);
```

Les appels système

L'Appel Système wait()

Les fonctions **WEXITSTATUS** et **WIFEXITED**

En résumé :

WIFEXITED(wstatus) → "Est-ce que c'était une fin normale ?"

(oui/non)

WEXITSTATUS(wstatus) → "Quel était le vrai code de sortie ?"

(uniquement si la réponse à 1 est "oui")

Les appels système

L'Appel Système wait()

Les fonctions **WEXITSTATUS** et **WIFEXITED**

Exercice :

Écris un programme qui :

1. Crée **un seul processus fils**
2. Le fils affiche son PID et se termine avec le code 7
3. Le père attend le fils
4. Le père analyse et affiche :
 - Le PID du fils
 - Si la terminaison était normale
 - Le code de retour

Les appels système

L'Appel Système wait()

Les fonctions **WEXITSTATUS** et **WIFEXITED**

Correction :

```
pid_fils = fork();

if (pid_fils == 0) {
    // Code exécuté par le fils
    printf("Je suis le fils, mon PID est : %d\n", getpid());
    exit(7); // Termine avec code 7
}
else if (pid_fils > 0) {
    // Code exécuté par le père
    printf("Je suis le père, mon PID est : %d\n", getpid());

    // Attente de la fin du fils
    wait(&status);
```

```
// Analyse du statut
printf("\nAnalyse du fils :\n");
printf(" PID du fils : %d\n", pid_fils);

if (WIFEXITED(status)) {
    printf(" Terminaison normale : OUI\n");
    printf(" Code de retour : %d\n", WEXITSTATUS(status));
} else {
    printf(" Terminaison normale : NON\n");
}
}
else {
    // Erreur de fork
    printf("Erreur lors de la création du processus fils\n");
    return 1;
}
```

Processus Orphelins



Processus dont le parent s'est terminé avant lui

Gestion automatique par le système :

- 1 Adoption automatique par init_task (PID 1)
- 2 Le PPID devient 1
- 3 Continue à s'exécuter normalement
- 4 Pas de problème de ressources

Lecture de fichier

on utilise un **pointeur de type FILE** et la fonction `fopen()`.

```
FILE *file = fopen("example.txt", "r");
```

Explications :

`FILE *file` : C'est un **pointeur vers un objet de type fichier**. Il contient toutes les informations nécessaires pour interagir avec le fichier.

`fopen("example.txt", "r")` :

Cette fonction ouvre le fichier "example.txt" en **mode lecture** ("r").

- Si l'ouverture réussit, **fopen()** retourne un pointeur vers le fichier.
- Si le fichier n'existe pas ou s'il y a une erreur, **fopen()** retourne **NULL**.

Ecriture dans un fichier

Envoyer des données

```
ssize_t bytes_written = write(fd, data, sizeof(data) - 1);
```

- **Détail des paramètres**

- **fd** : le descripteur obtenu avec `open()`
- **data** : pointeur vers les données (ex. une chaîne)
- **sizeof(data) - 1** : nombre d'octets à écrire (on ignore le `\0` final)

- **Retour (`ssize_t`)**

- **> 0** : nombre d'octets réellement écrits
- **-1** : erreur (descripteur invalide, disque plein...)

📌 `ssize_t` est un type signé : il peut stocker des tailles ou une valeur négative pour signaler une erreur.

Ouverture de fichier

```
int fd = open("example.txt", O_WRONLY | O_CREAT | O_TRUNC, 0666);
```

- **Que fait-on ?**
 - **O_WRONLY** → écriture seule
 - **O_CREAT** → crée le fichier s'il n'existe pas
 - **O_TRUNC** → vide le fichier s'il existe déjà
 - **0666** → permissions initiales (rw-rw-rw)
 - **Résultat**
 - **fd** : un petit entier positif (descripteur de fichier)
 - **-1** : échec (fichier inaccessible, erreur de permission...)
-  *Toujours vérifier la valeur de retour avant d'utiliser fd !*